The audit ledger: queries, the anchor, and the auditor's walk

TBK SPEC 40:2026, Version 1.0

The Tebako Project

Unrestricted
November 01, 2026
TBK SPEC 40:2026, Version 1.0
Tebako
© 2026 The Tebako Project. All rights reserved

Ribose

Contents

Foreword 4

1. The ledger and its one job 4

2. Reading rules (normative) 5

2.1 Line grammar 5

2.2 Damage is surfaced, never swallowed 5

2.3 The vocabulary (by the question it answers) 6

3. The query surface 6

3.1 Filters 7

3.2 Output 7

4. The anchor: tamper evidence 7

4.1 The chain 8

4.2 Seal records 8

4.3 Verification 9

5. The JSON contract (`audit_schema: 1`) 10

6. Named errors and exit codes 11

7. The auditor's walk 12

8. What this spec does not own 12

© 2026 The Tebako Project

All rights reserved. Unless otherwise specified, no part of this publication may be reproduced or utilized otherwise in any form or by any means, electronic or mechanical, including photocopying, or posting on the internet or an intranet, without prior written permission. Permission can be requested from the address below.

The Tebako Project

Suite 1, 8/F, New Henry House
10 Ice House Street
Central
Hong Kong

copyright@ribose.com
www.ribose.com

© 2026 The Tebako Project

Foreword

Status: SHIPPED. The event vocabulary predates this spec (the writers own their events, §2.3); this spec owns the READ contract and the anchor: the filter grammar, the output documents, the chain, the named errors — and the walk an auditor with no tebako knowledge follows to prove, offline, what ran on a host.

1. The ledger and its one job

Every tebako process writes one append-only line per decision worth auditing into a single host-local file:

FIGURE 1

```
$TEBAKO_HOME/journal.log          (TEBAKO_JAIL_JOURNAL
overrides the path)
```

The line shape is shared by both writers — the bootstrap (the loader's fetch / verify / mount / exec decisions) and `tfs::journal` (the driver's and preload's policy, VFS, overlay and decrypt decisions):

FIGURE 2

```
<unix seconds> event=<name> <key>=<value> …
```


© 2026 The Tebako Project

The ledger answers exactly one question — "what ran here, and why did it run?" — for one host, from one file, with no network and no other tebako artifact. It is not aggregated, not sharded, not rotated by the tools (an operator may rotate it; `--seal` before rotation is the discipline that keeps the seam auditable).

The read-only law. A query (`tebako audit` … without `--seal`) creates nothing, appends nothing, stamps nothing, migrates nothing. It runs on a host whose `~/.tebako` was written by an older tebako, and it must not move that store's layout forward on the way in. The single writing verb is `--seal`, and the only file it writes is the anchor (§4).

2. Reading rules (normative)

2.1 Line grammar

- The first space-delimited token is the timestamp: unix seconds, decimal. `0` is legitimate (a clock failure never fails the audited operation).
- Every following token is `key=value`, split at the FIRST =, so a value may itself contain = (URLs with query strings, base64).
- A token that carries no = CONTINUES the previous value, joined by one space. Writers record paths, and paths contain spaces: a legitimate `path=/a b/c` must not turn into damage. A leading token without = before any key exists is damage.
- `event=` is the required key: a line without it, or with an unreadable timestamp, is `malformed` (§2.2).
- A trailing newline terminates the record; a final record without one is still a record (the bytes are the record).
- Duplicate keys: the FIRST statement stands (§5's `fields_map_rule`) — a writer's second mention of a key never retroactively rewrites what it already said.

2.2 Damage is surfaced, never swallowed

A line that does not parse is a `malformed` record: it is counted, carries its 1-based line number, and appears in the JSON document's `malformed` array with its raw bytes. A ledger that quietly drops what it cannot read is worse than one that admits damage. A query's exit code stays 0 for malformed lines unless `--strict` is given (§6) — the default posture is "show me everything, including the damage", because an operator diagnosing a host should not have to guess what was hidden.

A malformed line is still a LINE: it occupies its position in the file and it is covered by the anchor's chain exactly like a readable one (a deleted line moves every later record,

© 2026 The Tebako Project

readable or not). "Record" and "line" are therefore interchangeable in the anchor's vocabulary (§4).

2.3 The vocabulary (by the question it answers)

The writers own their event names — this table is the census, and it exists so an auditor can phrase a filter. It is not exhaustive: an unknown `event=` value is a first-class record, filterable like any other (forward compatibility is the point of a ledger).

TABLE 1

question	events
What did we fetch, and from where?	`fetch`, `delta-fetch`, `mirror-materialized`, `payload-mirror-served`, `bundle-staged`, `bundle-runtime-warm`, `key-retrieval`
Did it verify, and against what?	`runtime-fetch-verified`, `runtime-index-verified`, `bundle-fetch-verified`, `bundle-fetch-unverified`, `payload-signature-trusted`, `bundle-trusted-root`, `bundle-trusted-forwarded`, `legacy-unsigned-accepted`, `unsigned-runtime-fetch`, `trust-bridge-java-gap`, `key-retrieval-failed`
What got installed, bound or retired?	`payload-installed`, `payload-uninstalled`, `runtime-own-install`, `runtime-own-shim`, `origin-rebind`, `trusted-key-removed`, `trust-ca-added`, `trust-ca-removed`, `registry-row-retired`, `registry-key-pinned`
What was mounted and composed?	`composition`, `overlay`, `decrypt`, `lazy-seed`, `lazy-fallback`, `slice-augment`, `slice-overlap`, `slice-skip`, `visibility`
What was resolved and executed?	`runtime-edge-resolved`, `provider-runtime-resolved`, `executable-edge-resolved`, `unlocked-pick`, `spawn-shim-note`
What did the policy decide?	`jail-deny`, `jail-allow`, `vfs-deny`, `vfs-write`, `lib-load`, `network-config`
What did a press decide?	`press-sign-opt-out`, `stale-registry-serve`, `edge-platform-skip`

3. The query surface

FIGURE 3

```
tebako audit [filters] [--count] [--json] [--strict]
```


© 2026 The Tebako Project

3.1 Filters

TABLE 2

flag	meaning
`--since <t>`	keep records with `ts >= t`. `t` is unix seconds or RFC 3339 UTC (`YYYY-MM-DDTHH:MM:SSZ`).
`--until <t>`	keep records with `ts <= t`. Same grammar.
`--event <e>[,<e>…]`	keep records whose `event=` is one of the listed names. Repeatable; the union of all occurrences. Exact match, never a prefix.
`--artifact <needle>`	keep records whose body (everything after the timestamp) CONTAINS `needle` — the artifact reference as the writer recorded it: a payload name, a version, a sha256, a URL fragment. Case-sensitive, documented as a substring so nobody mistakes it for a structured field.
`--tail <n>`	after filtering, keep only the last n records.

Filters compose with AND across flags, OR within `--event`. No filter means the whole ledger. A time filter's grammar is exact: local time is never inferred, because a silently shifted window is a silently wrong answer.

`--seal` takes NO filters (an anchor covers the whole file or it is not an anchor) and `--seal` with `--verify` is refused by name: one writes, the other checks.

3.2 Output

Default (line mode). The matching records, verbatim — the ledger's own bytes, in file order — so a reader can pipe them to `sha256sum` and compare against a seal. A one-line summary (`N of M record(s)`) goes to STDERR, never into the stream: the stdout of a query is a ledger excerpt, not a report about one.

`--count`. Exactly the match count on stdout, nothing else.

`--json`. The `audit_schema: 1` document (§5) on stdout alone; the version banner moves to stderr, like every other machine contract in this toolchain.

File order is preserved in every mode. A ledger's order IS evidence; re-sorting it would destroy the only thing the file cannot be rebuilt without.

4. The anchor: tamper evidence

An append-only file with no anchor proves nothing — anyone can edit a line, or delete a run's records entirely, and the file still looks like a ledger. The anchor is a chained digest,

© 2026 The Tebako Project

sealed on demand and verified on demand, stored BESIDE the journal it covers and named by it:

FIGURE 4

```
<journal>.seal          e.g. $TEBAKO_HOME/journal.log.seal
```

Beside, not inside: the journal stays a pure record stream, every reader of it keeps working, and an explicit `TEBAKO_JAIL_JOURNAL` override carries its seal along instead of splitting the pair across two homes.

4.1 The chain

FIGURE 5

```
h_0   = SHA-256("tebako-ledger-v1\n")
(32 bytes)
h_i   = SHA-256(h_{i-1} ‖ line_i)
(32 bytes)
```

where ‖ is byte concatenation of the raw 32-byte digest with the line's exact bytes INCLUDING its terminating newline. Any edit, any deletion, any insertion shifts every subsequent digest — the anchor's whole value is that it cannot be recomputed without re-sealing, and a re-seal is itself a new record (§4.3).

The terminator is normalized: a final record whose newline an editor added or stripped is the same ledger. That one cosmetic difference is the only edit the anchor does not report, and it is reported as nothing because it changes nothing an auditor reads.

4.2 Seal records

One line per seal, appended to the seal file:

FIGURE 6

```
<unix seconds> head=<sha256 hex> lines=<n> key=<keyid hex|->
sig=<hex|->
```

- head — the chain digest over the journal's first `lines` lines.

© 2026 The Tebako Project

- `lines` — how many lines the seal covers (the file's state at seal time; later lines are the unsealed tail).
- `key` / `sig` — the seal's authenticity. When a signing key is available (the machine's press-local key, spec 09), the canonical payload

  FIGURE 7

  ```
  tebako-ledger-seal-v1\n<head>\n<lines>\n<ts>\n
  ```

  is signed detached (OpenPGP — the one mechanism this project uses) and `sig` carries the armored signature hex-encoded. With no key, both fields are `-`.

The unsigned seal is honest about what it is. It detects edits (a changed line no longer hashes to the sealed head) but not forgery (somebody with write access can rewrite the log and re-seal). The verb says so on stderr every time it writes one:

FIGURE 8

```
tebako: WARNING: ledger sealed WITHOUT a signature — this
detects
edits, not forgery (configure a key with 'tebako keys')
```

A signed seal, verified against the trusted keyring or a pinned key, is the enterprise answer.

A seal that cannot be produced because the key store refuses is a LOUD unsigned seal, never a failed seal: an unanchorable ledger is worse than an unsigned anchor. A `TEBAKO_SIGN` that is not a boolean is a named usage error, never a silent choice.

4.3 Verification

FIGURE 9

```
tebako audit --verify [--json] [--strict]
```

- Recompute the chain over the WHOLE journal and compare it against the seals. The LAST seal speaks for the file; the earlier ones LOCATE the damage.
- Walking the seals in order: the newest seal whose recomputed chain still matches marks the end of the known-good prefix, and the first seal after it names the interval that changed. That interval — `[known_good+1, failing.lines]`, 1-based and

© 2026 The Tebako Project

inclusive — is what the report prints, because "records 411–812 changed" is actionable and "the head differs" is not.

- A seal covering more lines than the file holds is not a prefix mismatch but a REMOVAL: the report says records are gone, and names the same interval.
- Three outcomes: `sealed` (the last seal matches), `tampered` (it does not, or a signed seal's signature does not validate → exit 80), `unsealed` (no seal file: named, exit 74, with the remedy — "run `tebako audit --seal`").
- A signed seal's signature is checked against the trusted keyring and the pinned trust store. An unknown signer is reported as `unknown-key` and the outcome stays `sealed` (the chain matched) with the authenticity gap NAMED, never quietly upgraded. An invalid signature is `tampered`, whatever the chain says — a valid chain under a forged seal is exactly what a re-seal looks like.
- A seal file with an unparseable line is a finding (exit 80), never a skipped line: corrupting the anchor is an attacker's first move, so damage there is never noise.

5. The JSON contract (`audit_schema`: 1)

FIGURE 10

```
{
  "audit_schema": 1,
  "journal": "/home/u/.tebako/journal.log",
  "matched": 3,
  "total": 812,
  "records": [
    {"line": 12, "ts": 1761000000, "event": "fetch",
     "fields": {"origin": "…", "sha256": "…"}, "raw": "…"}
  ],
  "malformed": [{"line": 411, "raw": "…"}],
  "verify": {
    "outcome": "sealed",
    "seal_ts": 1761000123,
    "sealed_lines": 812,
    "head": "…",
    "tail": 0,
    "divergence": null,
    "signer": "abcdef0123456789",
    "signature": "valid",
    "detail": "the chain matches the signed seal of …"
  }
```


© 2026 The Tebako Project

```
    }
```

TABLE 3

field	meaning
`matched` / `total`	records selected by the filters, and lines the file holds
`records[].fields`	the record's k=v tokens as a map, first-wins on duplicate keys
`records[].raw`	the record's exact bytes without the newline
`malformed[]`	unparseable lines, always present (empty array when clean)
`verify`	`null` for a query; the object above for `--verify`; absent for `--seal`, whose document carries `seal` instead
`verify.divergence`	`{"from_line": N, "to_line": M}` when the outcome is `tampered`, else `null`
`seal` (on `--seal`)	`{path, ts, head, lines, signer}` — `signer` is `null` when unsigned

`records` is capped by `--tail` and by the filters; `total` always reports the file, so a filtered document can never be mistaken for the whole ledger. `audit_schema` is 1 forever: extensions add keys, never retype or rename one.

6. Named errors and exit codes

TABLE 4

code	name	meaning
80	`EX_TEBAKO_LEDGER`	`--verify` found a mismatch or an invalid seal signature; an unparseable seal record; or `--strict` with malformed journal lines
74	`EX_TEBAKO_IO`	the journal or seal cannot be read or written; a query on a host with no journal yet; `--verify` with no seal yet
64	`EX_USAGE`	unparseable filter or time, unknown flag, `--seal` with filters, `--seal` together with `--verify`

Every message names the file and the remedy — never "invalid input".

© 2026 The Tebako Project

7. The auditor's walk

The arc's acceptance, spelled out as the steps an auditor with no tebako knowledge performs. Each step is offline and needs only public artifacts plus this document:

1. `tebako audit --json > ledger.json` — the host's decisions.
2. `tebako audit --event fetch,runtime-fetch-verified --json` — every byte source and its verification verdict.
3. Take any sha256 from step 2 and check the artifact against the registry row and its attestation (`gh attestation verify`), then reproduce the payload image per spec 14's reproducible-press rule.
4. `tebako audit --verify` — the ledger's own integrity, against the seal the host wrote; or `tebako audit --seal` first on a host that never anchored.
5. Confirm the mount set (`--event composition,overlay,decrypt`) matches the package's declared dependency tree (`tebako closure`).
6. Archive `ledger.json` + the seal file; a later `--verify` on the archived pair proves the archive is what the host wrote.

8. What this spec does not own

- The event names and their field vocabularies — the writers own those (§2.3 is a census, not a registry).
- Rotation, retention, remote shipping: operator policy. The seal is the seam that makes rotation auditable, and that is all this spec says about it.
- The signature mechanism itself: spec 09 owns OpenPGP, the keyring and the pinned trust store; this spec only names where a seal's signature is checked.

© 2026 The Tebako Project